System Contracts & Integration Requirements

Meshaai — System Contracts & Integration Requirements

Unified Technical Contract & System Specifications for Backend & Frontend Integration
Document Version: `2.2.0`
Target Stack: Spring Boot 4 / Java 21 Modulith ↔ Web & Mobile Frontends

---

Table of Contents

1. Global Protocols & Communication Standards
   - 1.1 Base URLs & Environments
   - 1.2 Global Request Headers
   - 1.3 Monetary Representation Contract
   - 1.4 Global RFC 7807 Error Envelope
2. Customer Journey & Integration Contracts
   - End-to-End Customer Flowchart
   - 2.1 Anonymous Catalog Discovery & Currency Localization
   - 2.2 Delivery Zone & Serviceability Check
   - 2.3 Passwordless Authentication & Identity (Phone OTP)
   - 2.4 Customer Email & Password Login
   - 2.5 Session Rotation & Token Refresh
   - 2.6 Customer Profile & Address Book Management
   - 2.7 Real-Time Order Quote & Pricing Freeze
   - 2.8 Order Placement & Idempotency
   - 2.9 Payment Checkout Session Initiation
   - 2.10 Payment Return & Lightweight Order Status Polling
   - 2.11 Customer Order History
   - 2.12 Customer Order Cancellation
3. Payment Gateway & Webhook Integration Contracts
   - Payment Integration Architecture Flowchart
   - 3.1 Gateway Browser Redirect Callback
   - 3.2 Inbound Asynchronous Payment Webhook
   - 3.3 Authoritative Payment Verification Rules & Reconciliation Matrix
4. Admin & Operations Journey & Integration Contracts
   - Admin Operations Flowchart
   - 4.1 Admin Authentication & Session Lifecycle
   - 4.2 Customer & Device Session Administration
   - 4.3 Admin Order Search, Inspection & Lifecycle Transitions
   - 4.4 Catalog Presets, Modifiers & Price Book Management
   - 4.5 Foreign Exchange Rates Management
   - 4.6 Kitchens & Delivery Zones Management
   - 4.7 Delivery Fee Rules & Promotions Engine
   - 4.8 Platform Kill-Switches & Global Settings

5. System State Machines & Lifecycle Specifications
   - 5.1 Order Lifecycle State Machine
   - 5.2 Payment Transaction State Machine
6. Kitchen Operator Responsibilities & Dispatch Flow
7. Frontend Implementation Guidelines & UI Recommendations
   - 7.1 Authentication State Management & Interceptors
   - 7.2 Form Field Validation & Input Masking
   - 7.3 Order Status Polling Loop Strategy
   - 7.4 Comprehensive Error Code ↔ Toast Mapping Reference

---

1. Global Protocols & Communication Standards

1.1 Base URLs & Environments

- **Local Development:** `http://localhost:8080`
- **Swagger / OpenAPI Documentation:** `GET /docs` (Swagger UI), `GET /api-docs/public`, `GET /api-docs/admin`, `GET /api-docs/webhooks`

1.2 Global Request Headers

Header Name	Type	Requirement	Description
`Content-Type`	String	**Required** on POST/PUT/PATCH	Must always be `application/json`.
`Authorization`	String	**Required** on Protected Routes	Bearer token format: `Bearer <JWT_ACCESS_TOKEN>`.
`Idempotency-Key`	String (UUID / Unique string)	**Required** on `POST /api/v1/orders`	Unique client transaction key preventing duplicate order placement across network retries.
`X-Correlation-Id`	String (UUID)	Optional (Recommended)	Frontend-generated UUID for end-to-end request tracing. Echoed in all responses and problem envelopes.
`CF-IPCountry` / `X-Country-Code`	String (ISO-2)	Optional	Geo-IP header (e.g. `NG`, `US`, `GB`, `GH`) used for automatic visitor currency detection.

1.3 Monetary Representation Contract

- **Backend Storage & API Minor Units:** All monetary values sent in request bodies or calculated internally are positive integers in **minor units** (e.g. Nigerian Kobo: `50000` = ₦500.00, USD Cents: `250` = $2.50).
- **Localized Display Object (`PriceDto`):** Endpoints that return customer-facing prices output a structured price object:

```
{
  "currency": "NGN",
  "amount": 50000,
  "formatted": "₦500.00"
}
```

Frontend UI Suggestions

- **Currency Formatting Helper:** Implement a centralized client-side formatting function using `Intl.NumberFormat` (e.g. `formatMoney(amount: number, currency: string)`) to handle decimal conversion consistently across web and mobile. Never perform manual string concatenation of symbols and decimal points.
- **Price Display Components:** Always render the backend-supplied `formatted` string from `PriceDto` when available to guarantee locale consistency and currency symbol accuracy without client recalculation.

1.4 Global RFC 7807 Error Envelope

Whenever an error occurs (HTTP 4xx or 5xx), the backend returns a standardized problem detail payload:

```
{
  "type": "https://meshaai.com/errors/invalid-otp",
  "title": "Bad Request",
  "status": 400,
  "detail": "Invalid verification code. 3 attempt(s) remaining.",
  "instance": "/api/v1/auth/otp/verify",
  "code": "INVALID_OTP",
  "correlationId": "4c8f25b2-32a1-432d-96cf-712808c4a169"
}
```

> **Important**
>
> **Frontend Error Handling Rule:** The frontend must branch its UI/toast logic on the `code` property (which is stable and guaranteed), **never** by regex-matching the human-readable `detail` string.

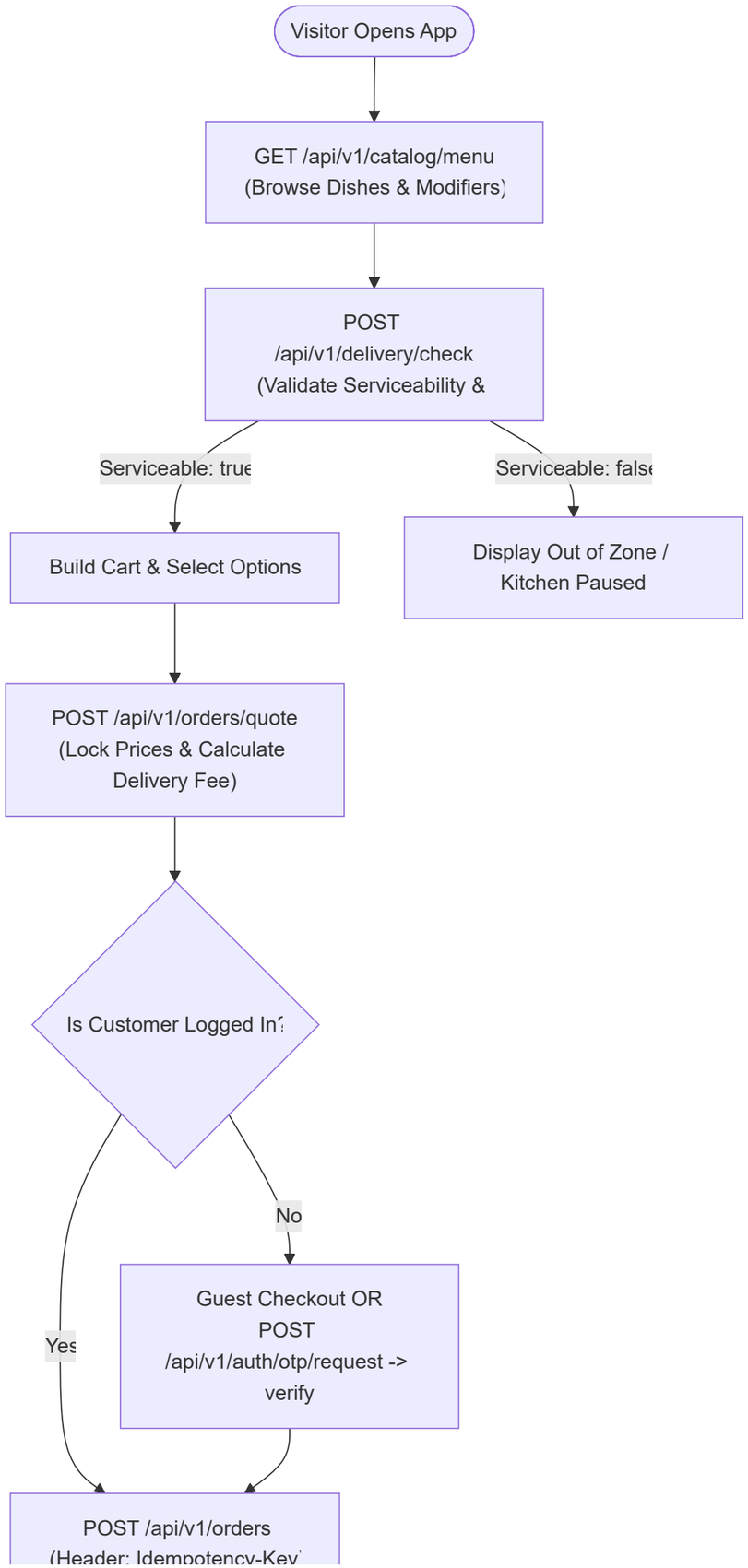
Frontend UI Suggestions

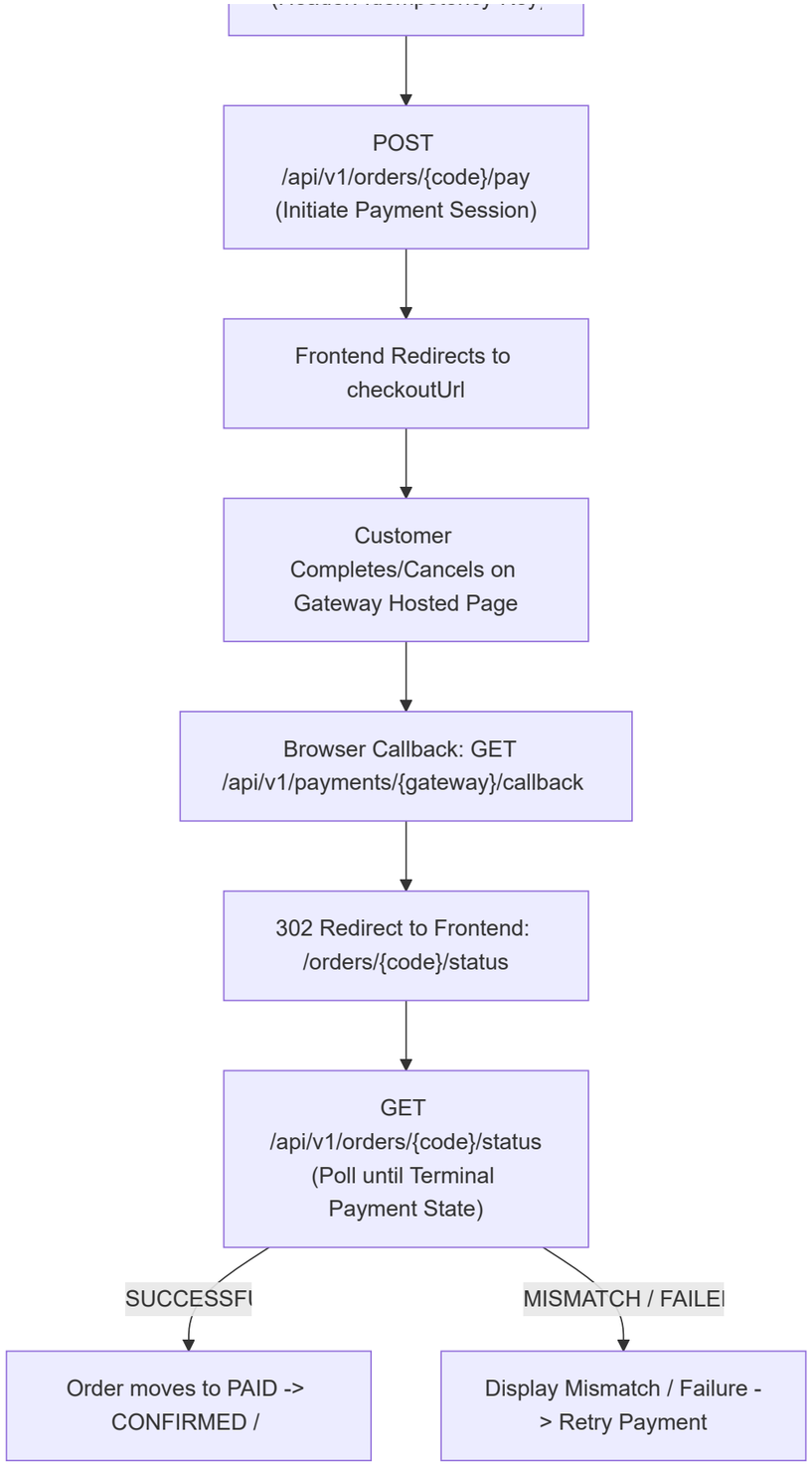
- **Global Error Interceptor:** In your HTTP client (e.g. Axios), intercept error responses and extract `error.response.data.code` to drive user-facing toast notifications using the lookup table in Section 7.4.
- **Graceful Fallback Alert:** If `code` is unrecognised or missing, display a gentle fallback toast (e.g. *"An unexpected error occurred. Please try again."*) while printing the `correlationId` to the developer console for support diagnostics.

---

2. Customer Journey & Integration Contracts

End-to-End Customer Flowchart


Visitor Opens App
GET /api/v1/catalog/menu
(Browse Dishes & Modifiers)
POST
/api/v1/delivery/check
(Validate Serviceability &
Serviceable: true
Serviceable: false
Build Cart & Select Options
Display Out of Zone /
Kitchen Paused
POST /api/v1/orders/quote
(Lock Prices & Calculate
Delivery Fee)
Is Customer Logged In?
No
Yes
Guest Checkout OR
POST
/api/v1/auth/otp/request ->
verify
POST /api/v1/orders
(Header: Idempotency-Key



2.1 Anonymous Catalog Discovery & Currency Localization

Requirement

Customers explore the menu, browse categories, view dishes, modifiers, and see accurate prices without creating an account or logging in.

API Contract

- **Endpoint:** `GET /api/v1/catalog/menu`
- **Query Parameters (Optional):** `?currency=USD` (explicit visitor currency override)
- **Headers (Optional):** `CF-IPCountry: US` (Cloudflare geo-IP header)
- **Expected Success Response (`200 OK`):**


```
{
  "currency": "NGN",
  "categories": [
    {
      "id": "jollof-bowls",
      "name": "Jollof Rice Bowls",
      "items": [
        {
          "id": "signature-smokey-jollof",
          "name": "Signature Smokey Jollof",
          "description": "Firewood-infused long grain jollof with fried plantains.",
          "available": true,
          "basePrice": {
            "currency": "NGN",
            "amount": 350000,
            "formatted": "₦3,500.00"
          },
          "modifierGroups": [
            {
              "id": "proteins",
              "name": "Choose Protein",
              "minSelections": 1,
              "maxSelections": 2,
              "options": [
                {
                  "id": "smoked-goat-meat",
                  "name": "Smoked Goat Meat",
                  "price": { "currency": "NGN", "amount": 120000, "formatted": "₦1,200.00" }
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```


Constraints & Edge Cases

Condition	HTTP Status	Error `code`	Frontend Handling
Unsupported `?currency=XYZ` override	`200 OK` (Fallback)	N/A	Backend gracefully falls back to base currency (`NGN`) or Geo-IP detected currency.
Item is sold out (`available: false`)	`200 OK`	N/A	Frontend disables "Add to Cart" and displays "Sold Out" overlay badge.

Frontend UI Suggestions

- **Currency Switcher:** A dropdown selector in the navigation bar listing supported currencies (`NGN` , `USD` , `GBP` , `EUR` , `CAD` , `GHS` , `KES`). Selecting an item sets the `?currency=` query parameter globally.
- **Modifier Modals:** Radio buttons for single-choice modifier groups (`minSelections=1, maxSelections=1`), and checkboxes with selection counters for multi-choice modifier groups (`maxSelections > 1`).

2.2 Delivery Zone & Serviceability Check

Requirement

Before placing an order, the customer checks if their delivery location is covered by a Meshaai kitchen, calculates the delivery fee, and checks whether the kitchen is open for immediate preparation or scheduled pre-orders.

API Contract

- **Endpoint:** `POST /api/v1/delivery/check`
- **Request Body:**

```
{
  "latitude": 9.0778,
  "longitude": 7.4819
}
```

- **Expected Success Response (`200 OK` — Inside Active Zone):**

```
{
  "serviceable": true,
  "kitchenId": 1,
  "kitchenName": "Meshaai Kitchen - Abuja Wuse 2",
  "distanceKm": 3.42,
  "deliveryFee": {
    "currency": "NGN",
    "amount": 80000,
    "formatted": "₦800.00"
  },
  "mode": "NOW",
  "ruleApplied": {
    "ruleId": 2,
    "ruleName": "Tier 2: 3-5km Standard",
    "type": "DISTANCE_BAND"
  }
}
```

- **Expected Success Response (`200 OK` — Outside Operating Hours / Pre-Order):**

```
{
  "serviceable": true,
  "kitchenId": 1,
  "kitchenName": "Meshaai Kitchen - Abuja Wuse 2",
  "distanceKm": 1.20,
  "deliveryFee": { "currency": "NGN", "amount": 50000, "formatted": "₦500.00" },
  "mode": "PRE_ORDER",
  "scheduledFor": {
    "utc": "2026-10-14T05:00:00Z",
    "local": "2026-10-14T06:00:00+01:00",
    "timezone": "Africa/Lagos"
  }
}
```

- **Expected Success Response (`200 OK` — Unserviceable Location):**

```
{
  "serviceable": false,
  "reason": "OUT_OF_ZONE"
}
```

Constraints & Edge Cases

Condition / Reason	Response Payload	Frontend Handling
`reason: "OUT_OF_ZONE"`	`serviceable: false`	Display banner: *"Sorry, we do not deliver to this location yet. Meshaai is currently serving within active kitchen zones in Abuja."*
`reason: "KITCHEN_PAUSED"`	`serviceable: false`	Display banner: *"The nearest kitchen is temporarily paused due to high order volume. Please check back shortly."*
`reason: "ORDERING_PAUSED"`	`serviceable: false`	Display modal: *"Online ordering is currently suspended across all kitchens for scheduled platform maintenance."*
Missing / null coordinates	`400 BAD_REQUEST`	Prompt user to enable browser location or select an address on map.

Frontend UI Suggestions

- **Address & Pin Picker:** Map interface or Google Places autocomplete where picking an address extracts `latitude` and `longitude` and immediately pings `/delivery/check`.
- **Fulfillment Pill Badge:** If `mode === "NOW"`, show green badge *"Delivering in 35-45 mins"*. If `mode === "PRE_ORDER"`, show yellow badge *"Kitchen opens at 06:00 AM (Pre-order now)"*.

2.3 Passwordless Authentication & Identity (Phone OTP)

Requirement

Customers authenticate without passwords. They enter their phone number and verify a 6-digit SMS OTP to automatically create an account or log into their existing profile.

Step 1: Request OTP

- **Endpoint:** `POST /api/v1/auth/otp/request`
- **Request Body:**

```
{
  "phone": "+2348039276495"
}
```

- **Expected Success Response (`200 OK`):**

```
{
  "message": "Verification code dispatched successfully",
  "expiresInSeconds": 300
}
```

Step 2: Verify OTP

- **Endpoint:** `POST /api/v1/auth/otp/verify`
- **Request Body:**

```
{
  "phone": "+2348039276495",
  "code": "760395",
  "name": "Tunde Bakare"
}
```

Parameter	Type	Required	Description
`phone`	`string`	**Yes**	Phone number in E.164 or national format.
`code`	`string`	**Yes**	6-digit OTP code received via SMS.
`name`	`string`	No	Full name of the customer. Captured during registration or guest checkout; saves/updates the name on the customer's profile if provided.

- **Expected Success Response (`200 OK`):**

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "9f8e7d6c-5b4a-3210-fedc-ba9876543210",
  "tokenType": "Bearer",
  "expiresInSeconds": 900,
  "customer": {
    "id": 104,
    "phone": "+2348039276495",
    "name": "Tunde Bakare",
    "email": null
  }
}
```

Constraints & Error Matrix

Scenario	Status	Error `code`	Frontend Behavior
Invalid phone format	`400`	`INVALID_PHONE_NUMBER`	Show inline input validation error under phone field.
Incorrect OTP code (Attempts 1 to 4)	`400`	`INVALID_OTP`	Highlight OTP input in red; display remaining attempts.
5 Consecutive Failed Attempts	`429`	`OTP_LOCKED`	Lock the OTP input for 15 minutes; show countdown timer.
OTP Expired (> 5 minutes)	`400`	`OTP_EXPIRED`	Clear OTP inputs; show *"Code expired. Request a new OTP."*
Request rate limit exceeded (> 3 req/min)	`429`	`RATE_LIMIT_EXCEEDED`	Disable "Resend Code" button; show retry countdown timer.
Customer blocked by Admin	`403`	`CUSTOMER_BLOCKED`	Show error dialog: *"Your account has been suspended. Please contact support."*
SMS Gateway (Termii) downtime or network timeout	`500`	`SMS_DELIVERY_FAILED`	Show alert: *"Unable to send SMS at this moment. Please try again in 1 minute."*

Frontend UI Suggestions

- **Phone Input:** Use a phone number input component with a country selector dropdown (defaulting to Nigeria `+234`).
- **Resend OTP Timer:** A 60-second countdown timer on the "Resend OTP" button to prevent client-side spamming.
- **Auto-Submit OTP:** Auto-submit the form as soon as the 6th digit is typed or pasted.

2.4 Customer Email & Password Login

Requirement

Registered customers who have attached an email and password to their profile can authenticate via credentials instead of waiting for SMS OTP.

API Contract

- **Endpoint:** `POST /api/v1/auth/login`

- **Request Body:**

```
{
  "email": "customer@example.com",
  "password": "CustomerSecret123!"
}
```

- **Expected Success Response (`200 OK`):**

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "4a5b6c7d-8e9f-0123-4567-89abcdef0123",
  "tokenType": "Bearer",
  "expiresInSeconds": 900,
  "customer": {
    "id": 104,
    "phone": "+2348039276495",
    "fullName": "Amina Bello",
    "email": "customer@example.com",
    "isNewCustomer": false
  }
}
```

Constraints & Edge Cases

Scenario	Status	Error `code`	Frontend Handling
Invalid credentials	`401`	`INVALID_CREDENTIALS`	Show error: *"Invalid email or password."*
Customer account blocked	`403`	`CUSTOMER_BLOCKED`	Show alert: *"Account restricted. Contact customer support."*

Frontend UI Suggestions

- **Password Toggle:** Eye icon to toggle visibility of password characters.
- **Quick Switch:** Link to switch back to *"Sign in with SMS OTP"* if the customer forgets their password.

2.5 Session Rotation & Token Refresh

Requirement

The customer's `accessToken` expires every 15 minutes. The frontend silently uses the 90-day `refreshToken` to acquire a new access token without interrupting the user's checkout experience.

API Contract

- **Endpoint:** `POST /api/v1/auth/session/refresh`
- **Request Body:**

```
{
  "refreshToken": "9f8e7d6c-5b4a-3210-fedc-ba9876543210"
}
```

- **Expected Success Response (`200 OK`):**

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
```

```
  "refreshToken": "1a2b3c4d-5e6f-7890-abcd-ef1234567890",
  "tokenType": "Bearer",
  "expiresInSeconds": 900
}
```

Constraints & Security Protocols

Scenario	Status	Error `code`	Frontend Security Action
Expired Refresh Token (> 90 days)	`401`	`SESSION_EXPIRED`	Clear local secure store; redirect to login.
Revoked Session (Admin kill)	`401`	`SESSION_REVOKED`	Purge tokens and force re-authentication.
Token Replay Attack (Re-sending an old, rotated token)	`401`	`SESSION_REVOKED`	**Backend auto-revokes the entire device session.** Frontend must wipe tokens immediately.

Frontend UI / Interceptor Implementation

- Set up an HTTP client interceptor (e.g. Axios Interceptor).
- Upon receiving a `401 UNAUTHORIZED` on any protected route, queue failed requests, call `/api/v1/auth/session/refresh`, update the in-memory token, and replay the queued requests seamlessly.

2.6 Customer Profile & Address Book Management

Requirement

Authenticated customers can register email/password credentials to their account, and save delivery locations for streamlined one-tap checkout.

Endpoints

- `POST /api/v1/me/account` — Register email and password credentials.
- `GET /api/v1/me/locations` — List saved delivery locations.
- `POST /api/v1/me/locations` — Save a new delivery location.

Register Account Credentials

- **Endpoint:** `POST /api/v1/me/account`
- **Headers:** `Authorization: Bearer <CUSTOMER_JWT>`
- **Request Body:**

```
{
  "email": "amina.bello@example.com",
  "password": "SecurePassword123!",
  "name": "Amina Bello"
}
```

- **Expected Success Response (`201 CREATED`):**

```
{
  "id": 104,
  "phone": "+2348039276495",
  "name": "Amina Bello",
  "email": "amina.bello@example.com"
}
```

Save Delivery Location

- **Endpoint:** `POST /api/v1/me/locations`
- **Headers:** `Authorization: Bearer <CUSTOMER_JWT>`
- **Request Body:**

```
{
  "label": "Apartment",
  "address": "Plot 124 Aminu Kano Crescent, Wuse 2, Abuja",
  "landmark": "Near Banex Plaza",
  "latitude": 9.0778,
  "longitude": 7.4819
}
```

- **Expected Success Response (`201 CREATED`):**

```
{
  "id": 12,
  "label": "Apartment",
  "address": "Plot 124 Aminu Kano Crescent, Wuse 2, Abuja",
  "landmark": "Near Banex Plaza",
  "latitude": 9.0778,
  "longitude": 7.4819,
  "createdAt": "2026-10-13T10:15:30Z"
}
```

Frontend UI Suggestions

- **Address Cards:** Saved locations displayed as selectable cards with a radio button indicating the active delivery destination.
- **Location Pin Confirmation:** Interactive map verifying the saved address coordinates with drag-and-drop pin accuracy.

2.7 Real-Time Order Quote & Pricing Freeze

Requirement

Before submitting an order, the frontend requests an authoritative quote. The backend:

1. Validates all line items, presets, and modifier options against the active catalog.
2. Evaluates the delivery address coordinates against physical kitchens and delivery fee rules.
3. Locks the calculated prices, subtotal, delivery fee, total, exchange rate, and assigned kitchen into a short-lived `quoteId` (valid for 15 minutes).

API Contract

- **Endpoint:** `POST /api/v1/orders/quote`
- **Request Body:**

```
{
  "lines": [
    {
      "presetId": "bread-2-eggs",
      "qty": 1,
      "modifiers": [
        {
          "modifierId": "extra-egg",
          "qty": 1
        }
```

```
      ]
    }
  ],
  "latitude": 9.0790,
  "longitude": 7.4830,
  "currency": "NGN"
}
```

- **Expected Success Response (`200 OK`):**

```
{
  "quoteId": "c4b3a210-9876-5432-10fe-dcba98765432",
  "kitchenId": 16,
  "kitchenName": "Meshaai Kitchen - Abuja Wuse 2",
  "currency": "NGN",
  "lines": [
    {
      "presetId": "bread-2-eggs",
      "name": "Bread + 2 Eggs",
      "qty": 1,
      "unitPrice": { "currency": "NGN", "amount": 200000, "formatted": "₦2,000.00" },
      "subtotal": { "currency": "NGN", "amount": 250000, "formatted": "₦2,500.00" },
      "modifiers": [
        {
          "modifierId": "extra-egg",
          "name": "Extra Egg",
          "qty": 1,
          "unitPrice": { "currency": "NGN", "amount": 50000, "formatted": "₦500.00" },
          "subtotal": { "currency": "NGN", "amount": 50000, "formatted": "₦500.00" }
        }
      ]
    }
  ],
  "subtotal": { "currency": "NGN", "amount": 250000, "formatted": "₦2,500.00" },
  "deliveryFee": { "currency": "NGN", "amount": 50000, "formatted": "₦500.00" },
  "total": { "currency": "NGN", "amount": 300000, "formatted": "₦3,000.00" },
  "mode": "NOW",
  "scheduledFor": null,
  "expiresAt": "2026-10-13T10:30:00Z"
}
```

Constraints & Validation Rules

Field / Rule	Constraint	Error `code`	Frontend Handling
`lines`	Must contain at least 1 line item (`@NotEmpty`).	`400 BAD_REQUEST`	Block checkout button until items are in cart.
`presetId`	Must exist and be active in catalog.	`400 PRESET_NOT_FOUND` / `PRESET_UNAVAILABLE`	Notify customer that item is no longer available.
`modifiers`	Modifiers must be allowed for the preset and within `maxQuantity` .	`400 INVALID_MODIFIERS`	Reset modifier selection in modal.
`latitude` / `longitude`	Coordinates outside all kitchen delivery radii.	`400 OUT_OF_ZONE`	Prompt customer with out-of-zone notice.
`currency`	If specified, must be supported by price book or exchange rates.	`400 INVALID_CURRENCY`	Fall back to `NGN` .

Frontend UI Suggestions

- **Live Cart Summary Drawer:** Sticky cart drawer updating line subtotals, modifier prices, and delivery fees in real time whenever quantity adjustments occur.

- **Quote Lock Countdown Badge:** Visual indicator showing remaining validity of the 15-minute quote lock before automatic recalculation.

2.8 Order Placement & Idempotency

Requirement

The customer commits the order from their locked quote. This creates an `Order` aggregate with status `PENDING_PAYMENT`. To prevent double charges from double clicks or network timeouts, the request **requires** an `Idempotency-Key` header.

API Contract

- **Endpoint:** `POST /api/v1/orders`
- **Headers:**
  - `Content-Type: application/json`
  - `Idempotency-Key: 7b8c9d0e-1f2a-3b4c-5d6e-7f8a9b0c1d2e` (**Required**)
  - `Authorization: Bearer <CUSTOMER_JWT>` (Optional; guests can place orders)
- **Request Body:**


```
{
  "quoteId": "c4b3a210-9876-5432-10fe-dcba98765432",
  "name": "Amina Bello",
  "phone": "+2348039276495",
  "location": "12 Adetokunbo Ademola Crescent, Wuse 2, Abuja",
  "landmark": "Near Banex Plaza",
  "note": "Please leave at the security gate"
}
```


- **Expected Success Response (`201 CREATED`):**


```
{
  "id": 501,
  "code": "MSH-K3F9A",
  "quoteId": "c4b3a210-9876-5432-10fe-dcba98765432",
  "customerName": "Amina Bello",
  "customerPhone": "+2348039276495",
  "deliveryAddress": "12 Adetokunbo Ademola Crescent, Wuse 2, Abuja",
  "deliveryLandmark": "Near Banex Plaza",
  "customerNote": "Please leave at the security gate",
  "latitude": 9.0790,
  "longitude": 7.4830,
  "kitchenId": 16,
  "kitchenName": "Meshaai Kitchen - Abuja Wuse 2",
  "status": "PENDING_PAYMENT",
  "deliveryMode": "NOW",
  "scheduledFor": null,
  "subtotal": { "currency": "NGN", "amount": 250000, "formatted": "₦2,500.00" },
  "deliveryFee": { "currency": "NGN", "amount": 50000, "formatted": "₦500.00" },
  "total": { "currency": "NGN", "amount": 300000, "formatted": "₦3,000.00" },
  "items": [
    {
      "presetId": "bread-2-eggs",
      "presetName": "Bread + 2 Eggs",
      "unitPrice": { "currency": "NGN", "amount": 200000, "formatted": "₦2,000.00" },
      "quantity": 1,
```

```
      "subtotal": { "currency": "NGN", "amount": 250000, "formatted": "₦2,500.00" },
      "modifiers": [
        {
          "modifierId": "extra-egg",
          "modifierName": "Extra Egg",
          "unitPrice": { "currency": "NGN", "amount": 50000, "formatted": "₦500.00" },
          "quantity": 1,
          "subtotal": { "currency": "NGN", "amount": 50000, "formatted": "₦500.00" }
        }
      ]
    }
  ],
  "createdAt": "2026-10-13T10:16:00Z"
}
```


Constraints & Scenarios

Scenario	Status	Error `code`	Description / Action
Quote older than 15 mins	`400`	`QUOTE_EXPIRED`	Quote has expired. Prompt customer to re-quote cart.
Missing `Idempotency-Key` header	`400`	`BAD_REQUEST`	Frontend must generate and pass a client UUID key.
Replayed identical `Idempotency-Key`	`201`	N/A	**Idempotent replay:** Returns the exact previously created `OrderResponse` without duplicate insertion.
Global ordering paused	`400`	`ORDERING_PAUSED`	Emergency platform kill-switch active.

Frontend UI Suggestions

- **Idempotency Key Generation:** Generate a unique client UUID upon opening the checkout form; persist across network retry attempts of the same order submission.
- **Button Debouncing:** Disable "Place Order" button and render a loading spinner immediately upon first click to prevent accidental double-submits.

2.9 Payment Checkout Session Initiation

Requirement

The customer proceeds to pay for their order. Calling this endpoint contacts the active payment gateway (e.g. Flutterwave), registers the transaction attempt, and returns the gateway's hosted `checkoutUrl`.

> **Tip**
>
> **Live Link Reuse Contract:** If called multiple times while the order has an active attempt (`INITIATED` or `PENDING`), the backend **reuses and returns the existing live checkout link** without charging or duplicating sessions. If a prior attempt failed or was abandoned, it increments the attempt counter and creates a new link.

API Contract

- **Endpoint:** `POST /api/v1/orders/{code}/pay`
- **Path Variable:** `code` (e.g. `MSH-K3F9A`)
- **Expected Success Response (`200 OK`):**


```
{
  "checkoutUrl": "https://checkout.flutterwave.com/v3/hosted/pay/425a7b8c9d0e1f",
  "reference": "MSH-K3F9A-ATTEMPT-1-7d8a9b0c",
```

```
  "gateway": "flutterwave"
}
```

Constraints & Edge Cases

Scenario	Status	Error `code`	Frontend Handling
Order already paid (`PAID` , `CONFIRMED` , `SCHEDULED` , `PREPARING`)	`400`	`ORDER_NOT_PAYABLE`	Order is already paid. Redirect directly to tracking page.
Order is cancelled	`400`	`ORDER_NOT_PAYABLE`	Cannot pay for a cancelled order.
Non-existent order code	`404`	`ORDER_NOT_FOUND`	Display error: *"Order not found."*
Currency not supported by gateway (e.g. CAD)	`200`	N/A	Backend converts CAD to fallback (`NGN`) using locked rate and passes NGN amount to gateway.

Frontend UI Suggestions

- **Session Storage Backup:** Persist `orderCode` and `reference` in browser `sessionStorage` before redirecting the customer window to Flutterwave's `checkoutUrl` .
- **Direct Redirection:** Trigger `window.location.href = checkoutUrl` immediately upon receiving the initiation response.

2.10 Payment Return & Lightweight Order Status Polling

Requirement

After the customer finishes checkout and returns to the store, the frontend lands on `/orders/{code}/status` and polls this lightweight endpoint every 2–3 seconds to detect when payment verification finalizes.

API Contract

- **Endpoint:** `GET /api/v1/orders/{code}/status`
- **Path Variable:** `code` (e.g. `MSH-K3F9A`)
- **Expected Success Response (`200 OK`):**

```
{
  "code": "MSH-K3F9A",
  "status": "CONFIRMED",
  "total": {
    "currency": "NGN",
    "amount": 300000,
    "formatted": "₦3,000.00"
  },
  "deliveryMode": "NOW",
  "scheduledFor": null,
  "paymentStatus": "SUCCESSFUL"
}
```

Polling Termination Contract

`paymentStatus`	Order `status`	Frontend Action
`INITIATED` or `PENDING`	`PENDING_PAYMENT`	**Keep polling** (Interval: 2-3 seconds, max 60s timeout). Show spinner: *"Verifying payment with bank..."*
`SUCCESSFUL`	`CONFIRMED` / `SCHEDULED`	**Stop polling.** Show celebration checkmark and redirect to order tracking screen.
`FAILED`	`PENDING_PAYMENT`	**Stop polling.** Show error modal: *"Payment declined. Please try another card or payment method."* Allow retry via `/pay` .
`ABANDONED`	`PENDING_PAYMENT`	**Stop polling.** Show notice: *"Payment was cancelled. You can retry anytime."*

`paymentStatus`	Order `status`	Frontend Action
`MISMATCH`	`PENDING_PAYMENT`	**Stop polling.** Show warning: *"Payment amount or currency mismatch. Our support team has been alerted."*

Frontend UI Suggestions

- **Pulsing Status Spinner:** Full-screen payment confirmation card with a pulsing loader: *"Confirming your payment with your bank..."*.
- **Celebration Handoff:** Green checkmark animation with order code on `paymentStatus === "SUCCESSFUL"`, transitioning automatically to live order tracking.
- **Graceful Timeout Fallback:** If polling exceeds 60 seconds without final resolution, provide a *"Check Status Again"* button along with an reassurance notice that an SMS confirmation will arrive shortly.

2.11 Customer Order History

Requirement

Authenticated customers can view their complete past order history, sorted chronologically with the newest orders first.

API Contract

- **Endpoint:** `GET /api/v1/me/orders`
- **Headers:** `Authorization: Bearer <CUSTOMER_JWT>`
- **Expected Success Response (`200 OK`):**


```
[
  {
    "id": 501,
    "code": "MSH-K3F9A",
    "quoteId": "c4b3a210-9876-5432-10fe-dcba98765432",
    "customerName": "Amina Bello",
    "customerPhone": "+2348039276495",
    "deliveryAddress": "12 Adetokunbo Ademola Crescent, Wuse 2, Abuja",
    "deliveryLandmark": "Near Banex Plaza",
    "customerNote": "Please leave at security",
    "latitude": 9.0790,
    "longitude": 7.4830,
    "kitchenId": 16,
    "kitchenName": "Meshaai Kitchen - Abuja Wuse 2",
    "status": "CONFIRMED",
    "deliveryMode": "NOW",
    "scheduledFor": null,
    "subtotal": { "currency": "NGN", "amount": 250000, "formatted": "₦2,500.00" },
    "deliveryFee": { "currency": "NGN", "amount": 50000, "formatted": "₦500.00" },
    "total": { "currency": "NGN", "amount": 300000, "formatted": "₦3,000.00" },
    "items": [
      {
        "presetId": "bread-2-eggs",
        "presetName": "Bread + 2 Eggs",
        "unitPrice": { "currency": "NGN", "amount": 200000, "formatted": "₦2,000.00" },
        "quantity": 1,
        "subtotal": { "currency": "NGN", "amount": 250000, "formatted": "₦2,500.00" },
        "modifiers": []
      }
    ],
    "createdAt": "2026-10-13T10:16:00Z"
```

```
  }
]
```

Frontend UI Suggestions

- **Order Cards with Status Chips:** Card-based list displaying human-readable order code, total formatted price, and colored status chips (`CONFIRMED` , `PREPARING` , `DELIVERED`).
- **Reorder Shortcut:** One-tap button populating the cart with identical items from past orders.

2.12 Customer Order Cancellation

Requirement

Customers can cancel their order **prior to preparation commencing** in the kitchen. If the order was already paid, the backend automatically flags the order for refund dispatch.

API Contract

- **Endpoint:** `POST /api/v1/orders/{code}/cancel`
- **Path Variable:** `code` (e.g. `MSH-K3F9A`)
- **Headers:** `Authorization: Bearer <CUSTOMER_JWT>` (Optional if guest provides `phone`)
- **Request Body (Optional for authenticated, Required for guest):**

```
{
  "phone": "+2348039276495",
  "reason": "Changed my mind on delivery timing"
}
```

- **Expected Success Response (`200 OK`):**

```
{
  "code": "MSH-K3F9A",
  "status": "CANCELLED",
  "message": "Order successfully cancelled.",
  "refundInitiated": true
}
```

Constraints & Edge Cases

Scenario	Status	Error `code`	Frontend Handling
Order in `PREPARING` , `OUT_FOR_DELIVERY` , or `DELIVERED`	`400`	`ORDER_CANNOT_BE_CANCELLED`	Kitchen has already started preparation. Cancellation blocked.
Guest phone does not match order record	`403`	`FORBIDDEN`	Show error: *"Phone number does not match order record."*
Order already cancelled or expired	`400`	`ORDER_ALREADY_CANCELLED`	Order is already in terminal state.

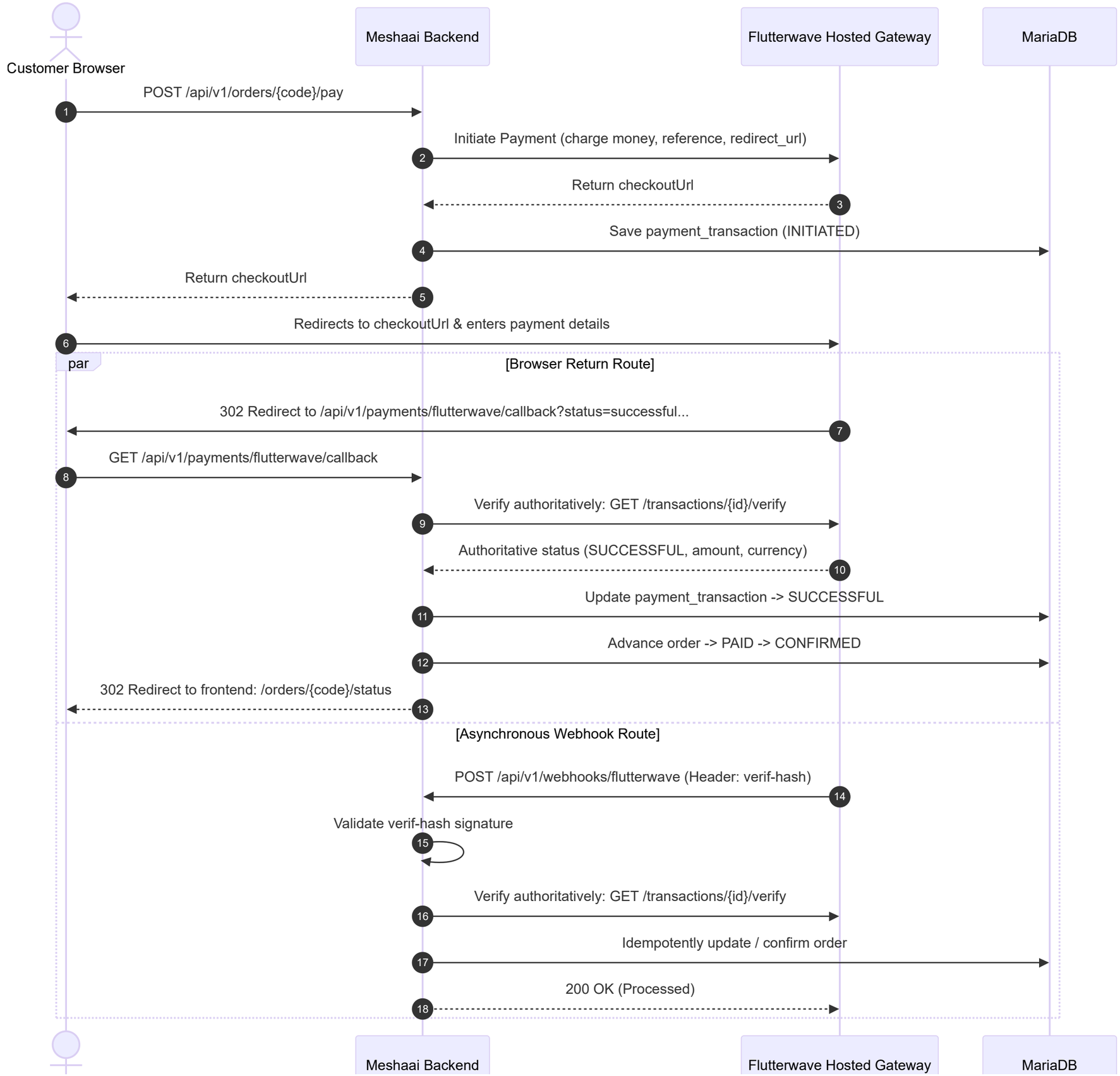
Frontend UI Suggestions

- **Destructive Confirmation Modal:** Red confirmation dialog warning the customer that cancellation will release their kitchen slot and initiate a refund.
- **Cancellation Reason Input:** Dropdown menu with common cancellation reasons and an optional comment box.

---

3. Payment Gateway & Webhook Integration Contracts

Payment Integration Architecture Flowchart


Customer Browser
Meshaai Backend
Flutterwave Hosted Gateway
MariaDB
POST /api/v1/orders/{code}/pay
1
Initiate Payment (charge money, reference, redirect_url)
2
Return checkoutUrl
3
Save payment_transaction (INITIATED)
4
Return checkoutUrl
5
Redirects to checkoutUrl & enters payment details
6
par
[Browser Return Route]
302 Redirect to /api/v1/payments/flutterwave/callback?status=successful...
7
GET /api/v1/payments/flutterwave/callback
8
Verify authoritatively: GET /transactions/{id}/verify
9
Authoritative status (SUCCESSFUL, amount, currency)
10
Update payment_transaction -> SUCCESSFUL
11
Advance order -> PAID -> CONFIRMED
12
302 Redirect to frontend: /orders/{code}/status
13
[Asynchronous Webhook Route]
POST /api/v1/webhooks/flutterwave (Header: verif-hash)
14
Validate verif-hash signature
15
Verify authoritatively: GET /transactions/{id}/verify
16
Idempotently update / confirm order
17
200 OK (Processed)
18
Meshaai Backend
Flutterwave Hosted Gateway
MariaDB

Customer Browser

3.1 Gateway Browser Redirect Callback

Requirement

When a customer finishes or cancels on the hosted gateway page, the gateway redirects the customer's browser back to our backend callback URL. The backend:

1. Parses gateway-specific parameters via the gateway port (`PaymentGateway.parseCallback`).
2. Authoritatively verifies the payment with the gateway servers.
3. Issues an HTTP `302 Found` redirect sending the browser to the frontend order status polling page (`/orders/{code}/status`).

API Contract

- **Endpoint:** `GET /api/v1/payments/{gateway}/callback`
- **Path Variable:** `gateway` (e.g. `flutterwave`)
- **Query Parameters:** Supplied by gateway (e.g. `status=successful&tx_ref=...&transaction_id=123456`)
- **Expected Success Response:** `HTTP 302 Found`
  - **Header:** `Location: http://localhost:3000/orders/MSH-K3F9A/status`

Frontend UI Suggestions

- **Callback Loading Screen:** Neutral loading transition state displayed while the browser callback finishes the 302 redirect to `/orders/{code}/status` .

3.2 Inbound Asynchronous Payment Webhook

Requirement

Flutterwave servers send signed JSON webhooks when payment status updates occur. The backend verifies the signature, authoritatively verifies the transaction against Flutterwave's verify API, and idempotently updates state.

API Contract

- **Endpoint:** `POST /api/v1/webhooks/{gateway}`
- **Path Variable:** `gateway` (e.g. `flutterwave`)
- **Headers:** `verif-hash: <SECRET_HASH>`
- **Request Body (Flutterwave Webhook Payload):**

```
{
  "event": "charge.completed",
  "data": {
    "id": 8923456,
    "tx_ref": "MSH-K3F9A-ATTEMPT-1-7d8a9b0c",
    "flw_ref": "FLW-MOCK-987654",
    "amount": 3000,
    "currency": "NGN",
    "charged_amount": 3000,
    "status": "successful"
  }
}
```

- **Expected Success Response (`200 OK`):**

```
{
  "status": "success",
  "message": "Webhook processed successfully"
}
```

Frontend UI Suggestions

- **Silent Background Updates:** The frontend does not interact directly with this endpoint. Instead, the polling loop on `/orders/{code}/status` detects when the webhook settles the transaction and automatically transitions the order tracking screen without requiring a manual page refresh.

3.3 Authoritative Payment Verification Rules & Reconciliation Matrix

Before updating any order state, `PaymentVerificationService` evaluates the authoritative gateway verification against the recorded transaction:

Gateway State	Currency Match	Amount Comparison	Resulting Transaction Status	Order Action	Event Published
`successful`	Matches	`received == expected`	`SUCCESSFUL`	Moves to `PAID` → `CONFIRMED` (or `SCHEDULED`)	`PaymentSucceededEvent`
`successful`	Matches	`received > expected`	`SUCCESSFUL`	Moves to `PAID` → `CONFIRMED` (overpayment accepted)	`PaymentSucceededEvent` AND `PaymentMismatchedEvent` (Alert)
`successful`	Matches	`received < expected`	`MISMATCH`	Order remains `PENDING_PAYMENT` (underpaid)	`PaymentMismatchedEvent` (Admin alert)
`successful`	**Differs**	Any	`MISMATCH`	Order remains `PENDING_PAYMENT` (wrong currency)	`PaymentMismatchedEvent` (Admin alert)
`failed`	Any	Any	`FAILED`	Order remains `PENDING_PAYMENT`	None
`cancelled`	Any	Any	`ABANDONED`	Order remains `PENDING_PAYMENT`	None
Re-delivered Webhook	Already `SUCCESSFUL`	Any	`SUCCESSFUL`	Idempotently republishes event in case order update failed	`PaymentSucceededEvent`

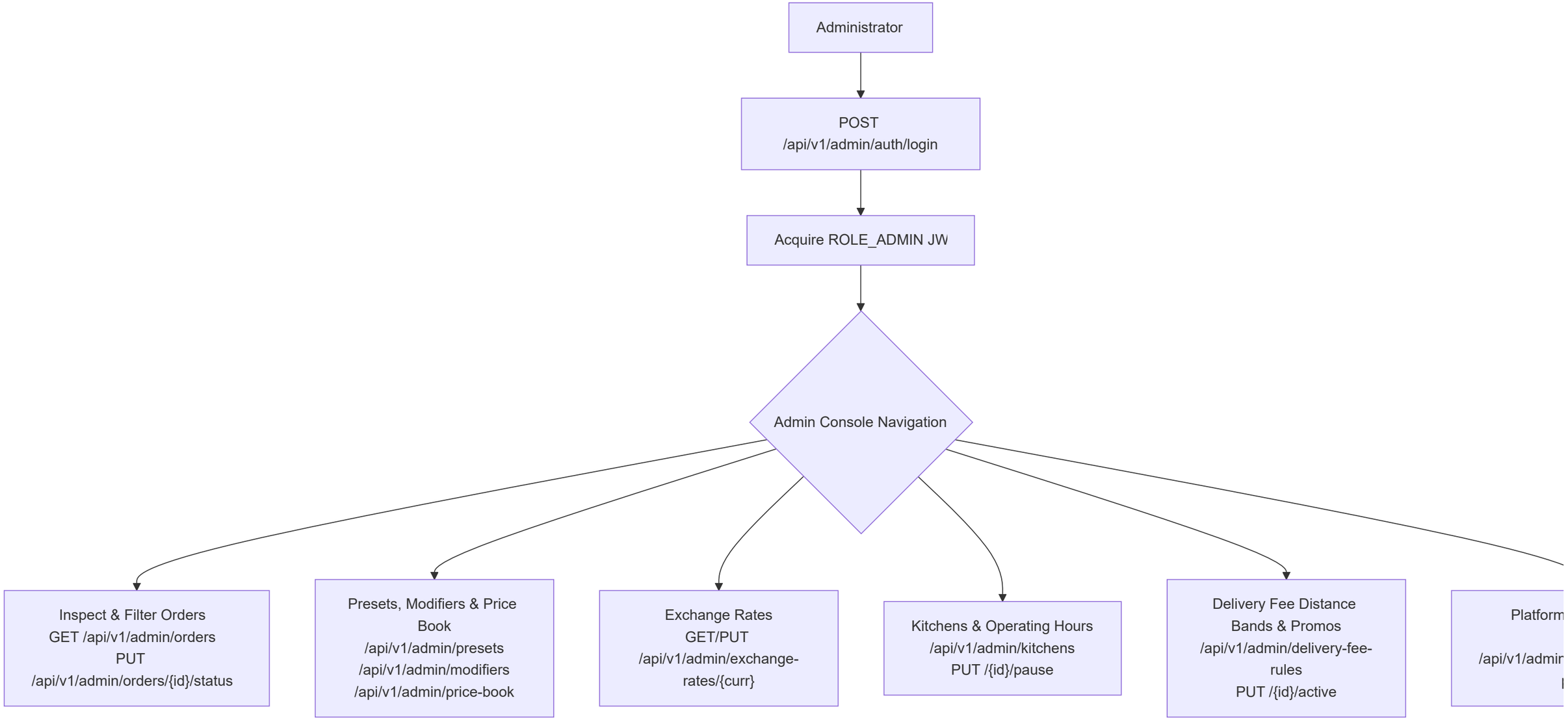
Frontend UI Suggestions

- **Mismatch Alert Banner:** If `paymentStatus === "MISMATCH"`, render an alert banner informing the customer of the amount discrepancy and providing a direct button to chat with support or retry payment with the full required amount on a new checkout attempt.
- **Overpayment Notice:** On the post-payment return screen, display a positive confirmation banner informing the customer that their order is confirmed and an excess refund or credit memo has been flagged for administrative review.

---

4. Admin & Operations Journey & Integration Contracts

Admin Operations Flowchart



4.1 Admin Authentication & Session Lifecycle

API Contract

- **Endpoint:** `POST /api/v1/admin/auth/login`
- **Request Body:**

```
{
  "email": "admin@meshaai.com",
  "password": "SecureAdminPassword123!"
}
```

- **Expected Success Response (`200 OK`):**

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "48b2a3c4-d5e6-7890-abcd-ef0123456789",
```

```
  "tokenType": "Bearer",
  "expiresInSeconds": 900,
  "admin": {
    "id": 1,
    "email": "admin@meshaai.com",
    "fullName": "System Administrator"
  }
}
```


Frontend UI Suggestions

- **Admin Login Form:** Email and password inputs with remember-me preference and dedicated admin branding.

4.2 Customer & Device Session Administration

Endpoints

- `GET /api/v1/admin/customers?page=0&size=20&search=0803` — Paginated customer list.
- `GET /api/v1/admin/customers/{id}` — Customer details, addresses, and device sessions.
- `PUT /api/v1/admin/customers/{id}/block` — Block customer (immediately denies all OTP requests and revokes sessions).
- `PUT /api/v1/admin/customers/{id}/unblock` — Unblock customer.
- `POST /api/v1/admin/customers/{id}/sessions/revoke-all` — Force logout across all customer devices.

Frontend UI Suggestions

- **Customer Status Badge:** Green chip for *"Active"*, Red chip for *"Blocked"*.
- **Block Confirmation Modal:** Destructive action dialog requiring explicit confirmation before blocking a customer.
- **Active Devices List:** List showing active device sessions with a *"Revoke All Devices"* one-tap button.

4.3 Admin Order Search, Inspection & Lifecycle Transitions

Search & Filter Orders

- **Endpoint:** `GET /api/v1/admin/orders`
- **Headers:** `Authorization: Bearer <ADMIN_JWT>`
- **Query Parameters:**
  - `code` (e.g. `MSH-K3F9A`)
  - `phone` (e.g. `+2348039276495`)
  - `status` (e.g. `CONFIRMED`, `PREPARING`, `OUT_FOR_DELIVERY`)
  - `kitchenId` (e.g. `16`)
  - `fromDate` (ISO-8601, e.g. `2026-10-01T00:00:00Z`)
  - `toDate` (ISO-8601, e.g. `2026-10-14T23:59:59Z`)
  - `page`, `size` (Pagination)
- **Expected Success Response (`200 OK`):** Spring Data Page of `OrderResponse`.

Get Order by ID

- **Endpoint:** `GET /api/v1/admin/orders/{id}`

- **Headers:** `Authorization: Bearer <ADMIN_JWT>`
- **Expected Success Response (`200 OK`):** Full `OrderResponse` payload including all line items, snapshot pricing, and modifiers.

Perform Administrative Order Status Transition

- **Endpoint:** `PUT /api/v1/admin/orders/{id}/status`
- **Headers:** `Authorization: Bearer <ADMIN_JWT>`
- **Request Body:**

```
{
  "status": "PREPARING",
  "reason": "Kitchen ticket received and cooking commenced by Chef Ibrahim"
}
```

- **Expected Success Response (`200 OK`):** Updated `OrderResponse`.

Frontend UI Suggestions

- **Status Filter Tabs:** Tabs across the top of the table for quick filtering by status (`ALL`, `PENDING_PAYMENT`, `CONFIRMED`, `PREPARING`, `OUT_FOR_DELIVERY`, `DELIVERED`, `CANCELLED`).
- **Audit Timeline Modal:** Interactive vertical timeline showing each status transition, actor identifier, timestamp, and transition note.

4.4 Catalog Presets, Modifiers & Price Book Management

All catalog mutations atomically produce a new immutable versioned file (`data/catalog/catalog.vN.json`) and hot-swap memory caches without downtime.

Preset Endpoints

- `GET /api/v1/admin/presets` — List all presets including archived ones.
- `POST /api/v1/admin/presets` — Create new preset combo (`PresetCreateRequest`).
- `PUT /api/v1/admin/presets/{id}` — Update preset details (`PresetUpdateRequest`).
- `POST /api/v1/admin/presets/{id}/archive?archived=true` — Archive/unarchive preset.
- `DELETE /api/v1/admin/presets/{id}` — Archive preset (soft-delete).

Modifier Endpoints

- `GET /api/v1/admin/modifiers` — List all modifiers including archived ones.
- `POST /api/v1/admin/modifiers` — Create new modifier add-on (`ModifierCreateRequest`).
- `PUT /api/v1/admin/modifiers/{id}` — Update modifier details (`ModifierUpdateRequest`).
- `POST /api/v1/admin/modifiers/{id}/archive?archived=true` — Archive/unarchive modifier.
- `DELETE /api/v1/admin/modifiers/{id}` — Archive modifier (soft-delete).

Price Book Endpoints

- `GET /api/v1/admin/price-book` — View explicit currency price book overrides.
- `PUT /api/v1/admin/price-book` — Upsert price book entries:

```
{
  "entries": {
    "USD": {
      "bread-2-eggs": 250,
      "extra-egg": 60
    }
```

```
    }
  }
```

Frontend UI Suggestions

- **Quick-Toggle Table Switch:** An on/off switch in the admin inventory table that immediately dispatches availability changes with optimistic UI updates.
- **Archive Confirmation Modal:** Soft-delete confirmation dialog explaining presets are archived rather than permanently removed.

4.5 Foreign Exchange Rates Management

Requirement

Admins inspect and update preset currency exchange rates against the base currency (`NGN`). Updating an exchange rate evicts pricing caches immediately.

Endpoints

- `GET /api/v1/admin/exchange-rates` — List all configured exchange rates.
- `PUT /api/v1/admin/exchange-rates/{currency}` — Update rate multiplier for currency.

Update Exchange Rate Request Body:

```
{
  "rate": 0.00065000
}
```

- **Constraint:** `rate` must be strictly positive (`> 0.00000001`) with at most 8 decimal places.

Frontend UI Suggestions

- **Exchange Rate Input Mask:** Input allowing up to 8 decimal places with automatic preview conversion (e.g. `1 NGN = 0.00065000 USD`).

4.6 Kitchens & Delivery Zones Management

Endpoints

- `GET /api/v1/admin/kitchens` — List all kitchens.
- `GET /api/v1/admin/kitchens/{id}` — Get kitchen by ID.
- `POST /api/v1/admin/kitchens` — Create new kitchen (`KitchenCreateRequest`).
- `PUT /api/v1/admin/kitchens/{id}` — Update kitchen details and hours (`KitchenUpdateRequest`).
- `PUT /api/v1/admin/kitchens/{id}/pause?paused=true` — Toggle temporary operational pause.
- `DELETE /api/v1/admin/kitchens/{id}` — Deactivate kitchen from routing.

Create Kitchen Request Body:

```
{
  "name": "Meshaai Kitchen - Abuja Wuse 2",
  "latitude": 9.0778,
  "longitude": 7.4819,
  "timezone": "Africa/Lagos",
```

```
  "openTime": "06:00:00",
  "closeTime": "19:00:00",
  "radiusKm": 7.0,
  "operatorName": "Ibrahim Wuse Ops",
  "operatorPhone": "+2348039276495",
  "active": true,
  "paused": false
}
```

Frontend UI Suggestions

- **Time Pickers:** 24-hour time pickers enforcing `closeTime > openTime`.
- **Coverage Map Circle:** Interactive map displaying the service radius circle around the kitchen's latitude and longitude coordinates.
- **Emergency Pause Toggle:** Prominent pause switch on the kitchen card with yellow warning badge.

4.7 Delivery Fee Rules & Promotions Engine

Endpoints

- `GET /api/v1/admin/delivery-fee-rules` — List all delivery fee rules.
- `GET /api/v1/admin/delivery-fee-rules/{id}` — Get rule by ID.
- `POST /api/v1/admin/delivery-fee-rules` — Create fee rule (`DeliveryFeeRuleCreateRequest`).
- `PUT /api/v1/admin/delivery-fee-rules/{id}` — Update fee rule (`DeliveryFeeRuleUpdateRequest`).
- `PUT /api/v1/admin/delivery-fee-rules/{id}/active?active=true` — Enable/disable rule.
- `DELETE /api/v1/admin/delivery-fee-rules/{id}` — Soft delete rule.

Create Fee Rule Request Body:

```
{
  "name": "0-3km Standard Band",
  "ruleType": "DISTANCE_BAND",
  "priority": 10,
  "minDistanceKm": 0.0,
  "maxDistanceKm": 3.0,
  "amountMinor": 50000,
  "currency": "NGN",
  "kitchenId": 16,
  "active": true
}
```

Frontend UI Suggestions

- **Rule Type Selector:** Segmented control switching between *"Distance Band (Per Kitchen)"* and *"Promotional Time Window"*.
- **Priority Indicator:** Visual ranking chip showing rule priority (`10` vs `100`).

4.8 Platform Kill-Switches & Global Settings

Endpoints

- `GET /api/v1/admin/settings` — List all system parameters.
- `GET /api/v1/admin/settings/{key}` — Get specific setting.
- `PUT /api/v1/admin/settings/{key}` — Update parameter value.
- `PUT /api/v1/admin/settings/ordering-pause?paused=true` — Emergency master toggle pausing platform-wide order intake.

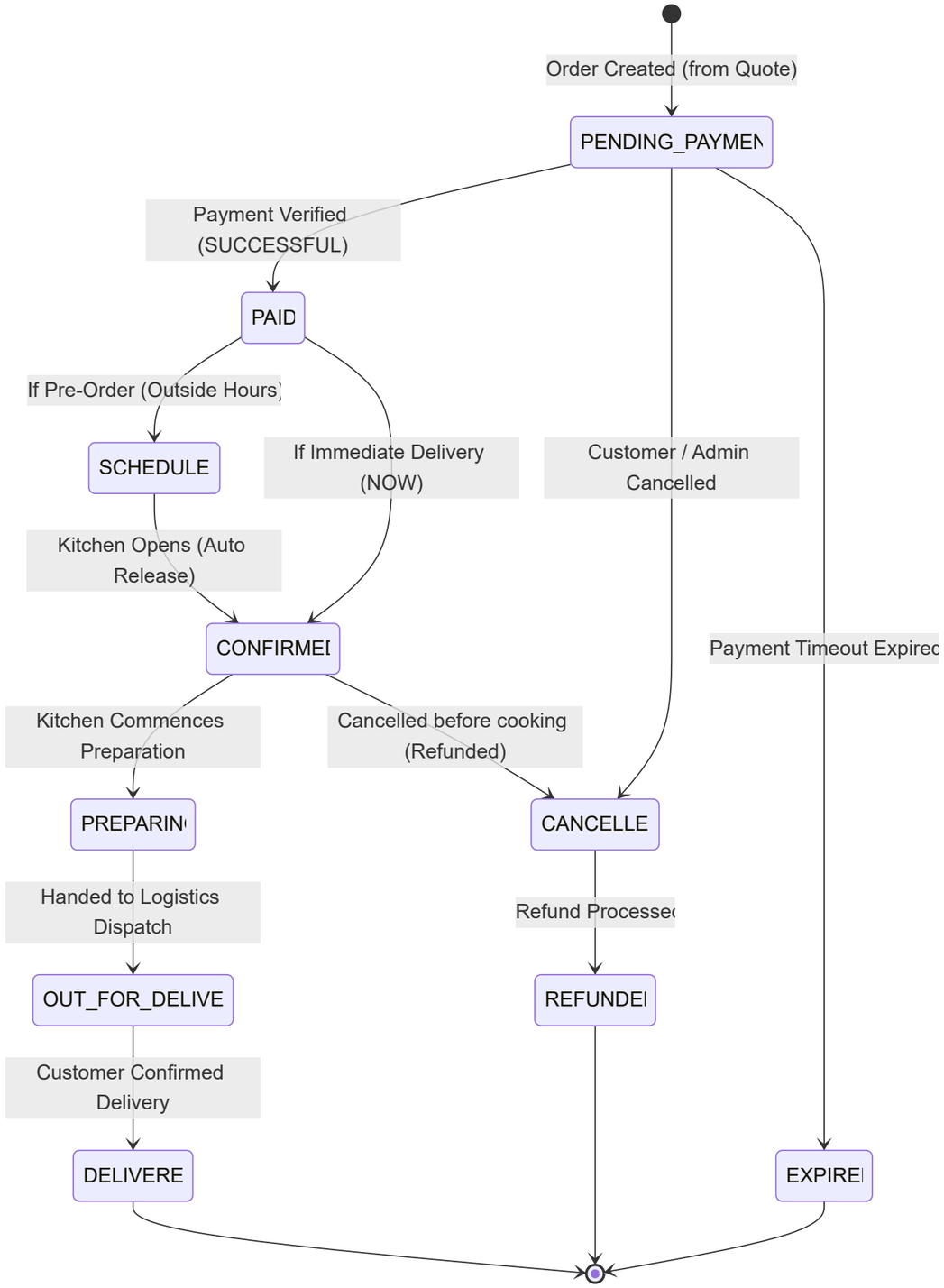
Frontend UI Suggestions

- **Master Emergency Banner:** High-visibility toggle switch in the admin header with a warning dialogue before pausing global operations.

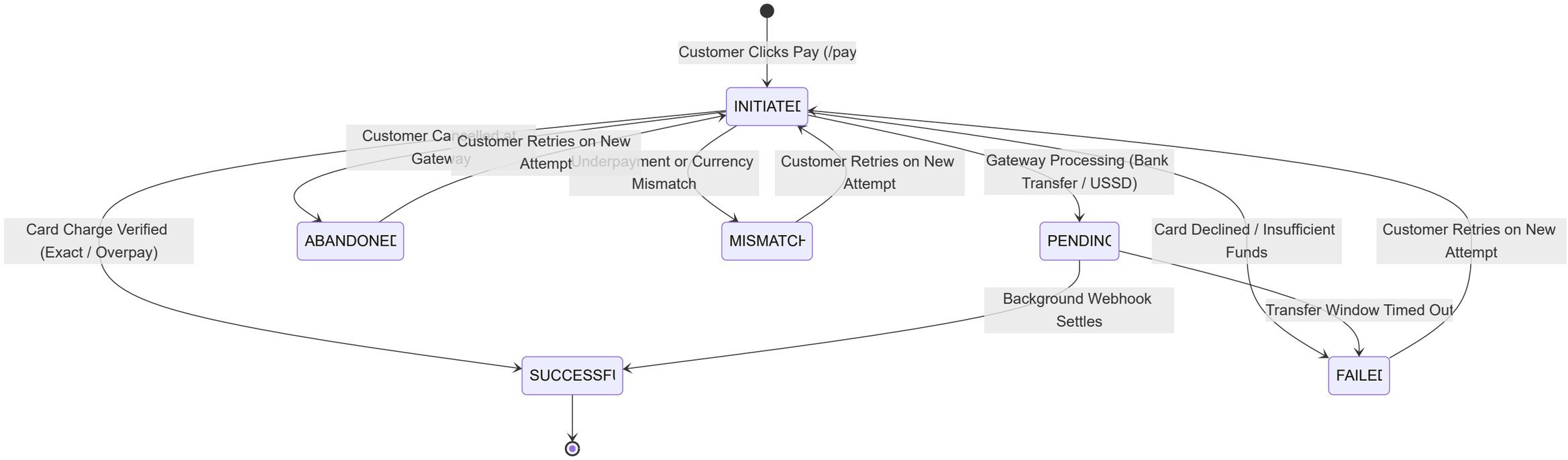
---

5. System State Machines & Lifecycle Specifications

5.1 Order Lifecycle State Machine



5.2 Payment Transaction State Machine



6. Kitchen Operator Responsibilities & Dispatch Flow

1. **Order Alert SMS:** The kitchen operator receives an instant SMS notification on their designated `operatorPhone` whenever a new order enters `CONFIRMED` in their kitchen zone.
2. **Kitchen Preparation Checklist:** When cooking begins, the operator advances the order to `PREPARING` via the operator interface.
3. **Dispatch Handoff:** Once sealed and packed, the operator hands the package to dispatch and advances status to `OUT_FOR_DELIVERY`.
4. **Emergency Pause:** If a kitchen experiences equipment failure or an overwhelming rush, the operator contacts the admin or triggers `PUT /api/v1/admin/kitchens/{id}/pause?paused=true` to temporarily pause intake while adjacent kitchens cover border zones.

Frontend UI Suggestions

- **Mobile-First Kitchen Dashboard:** Build a high-contrast, touch-optimized tablet/mobile view featuring large action buttons (`START PREPARING`, `DISPATCH ORDER`) that kitchen staff can easily tap while wearing gloves.
- **Audible New Order Chime:** Play an audio chime alert upon receiving incoming `CONFIRMED` orders to ensure kitchen staff are alerted immediately even if the tablet screen is dimmed.
- **Emergency Kitchen Pause Modal:** Provide a high-visibility pause toggle on the kitchen board with a confirmation dialog to prevent accidental service disruption.

---

7. Frontend Implementation Guidelines & UI Recommendations

7.1 Authentication State Management & Interceptors

```
interface AuthState {
  isAuthenticated: boolean;
  accessToken: string | null;
  refreshToken: string | null;
  customer: CustomerProfile | null;
}
```

- Store `refreshToken` in secure storage (`HttpOnly` cookie or secure device keychain).

- Store `accessToken` in memory.
- Implement an HTTP client interceptor (e.g. Axios): upon encountering a `401 UNAUTHORIZED`, queue outbound calls, invoke `/api/v1/auth/session/refresh`, replace the in-memory token, and replay queued requests.

7.2 Form Field Validation & Input Masking

Form Field	Recommended Frontend Component / Mask
Phone Number	Phone input with international flag selector; auto-formats to E.164 (e.g. `+234 803 927 6495`).
OTP Input	6 individual numeric input boxes with auto-focus advance and paste support.
Time Pickers	24-hour time selector format `HH:mm:ss`.
Coordinates	Google Places / Map pin picker outputting latitude and longitude decimals.
Minor Unit Amounts	Formats minor units (e.g. `250000`) into localized currency strings (`₦2,500.00`).

7.3 Order Status Polling Loop Strategy

When redirecting from the payment gateway to `/orders/{code}/status`:

1. Start a timer interval querying `GET /api/v1/orders/{code}/status` every 2.5 seconds.
2. If `paymentStatus === "SUCCESSFUL"`, immediately cancel timer, trigger celebration animation, and route to order tracking.
3. If `paymentStatus === "FAILED"` or `"ABANDONED"`, stop timer and render a retry payment banner.
4. If `paymentStatus === "MISMATCH"`, stop timer and present support alert dialog.
5. If no terminal state is reached within 60 seconds, display: *"We are still confirming your transaction with your bank. You will receive an SMS confirmation shortly."*

7.4 Comprehensive Error Code ↔ Toast Mapping Reference

Backend Error `code`	HTTP Status	Recommended User-Facing Toast / Alert Message
`INVALID_PHONE_NUMBER`	400	"Please enter a valid phone number including country code."
`INVALID_OTP`	400	"Incorrect verification code. Please check and try again."
`OTP_EXPIRED`	400	"Verification code has expired. Please request a new code."
`OTP_LOCKED`	429	"Too many failed attempts. Your account is temporarily locked for 15 minutes."
`RATE_LIMIT_EXCEEDED`	429	"Too many requests. Please wait a moment before trying again."
`CUSTOMER_BLOCKED`	403	"Your account has been restricted. Please contact Meshaai customer support."
`SMS_DELIVERY_FAILED`	500	"Unable to send SMS at this moment. Please try again in 1 minute."
`OUT_OF_ZONE`	200 / 400	"Delivery is not currently available at this location."
`KITCHEN_PAUSED`	200	"The kitchen serving your area is temporarily paused. Please check back shortly."
`ORDERING_PAUSED`	200 / 400	"Ordering is temporarily closed for maintenance. Please check back soon."
`QUOTE_EXPIRED`	400	"Your pricing quote has expired (15-min limit). We've refreshed your cart."
`PRESET_NOT_FOUND` / `PRESET_UNAVAILABLE`	400	"One or more items in your cart is currently sold out or unavailable."
`INVALID_MODIFIERS`	400	"Invalid modifier options selected for this meal item."
`ORDER_NOT_PAYABLE`	400	"Payment cannot be initiated. This order is already paid or cancelled."
`ORDER_CANNOT_BE_CANCELLED`	400	"Your food is already being prepared in the kitchen and cannot be cancelled."
`ORDER_ALREADY_CANCELLED`	400	"This order has already been cancelled."
`INVALID_EXCHANGE_RATE`	400	"Foreign currency exchange rate unavailable for this checkout."
`INVALID_HOURS`	400	"Kitchen closing time must be later than the opening time."

Backend Error `code`	HTTP Status	Recommended User-Facing Toast / Alert Message
`INVALID_CURRENCY`	400	"Invalid currency code. Please select a valid 3-letter currency code (e.g. NGN, USD)."
`SESSION_EXPIRED` / `SESSION_REVOKED`	401	"Your session has expired. Please log in again."
`UNAUTHORIZED`	401	"Authentication required to perform this action."
`FORBIDDEN`	403	"You do not have permission to access this resource."